



Rapport de Projet

Operation Research

CUVILLIEZ Fiona

Master 1 Informatique

9 mai 2026

Table des matières

1	Introduction du problème	2
2	Structure de données	3
2.1	Représentation du graphe	3
2.2	Graphe résiduel	3
3	Algo pour construire un flot max	5
3.1	Flot maximal et coupe minimale	5
3.1.1	Flot maximal	5
3.1.2	Min Cut	5
3.2	Recherche de chemin augmentant avec BFS	6
3.3	Algorithme de Ford-Fulkerson	6
3.3.1	Complexité	6
4	Min Cost Flow par Repeated Augmenting Paths	8
4.1	Version avec Bellman-Ford et coûts négatifs	8
4.1.1	Principe de Bellman-Ford	8
4.1.2	Cycles négatifs	9
4.1.3	Fonctionnement du Min Cost Flow	9
4.2	Avec Dijkstra et renormalisation des coûts	9
5	Détection de cycles négatifs	10
6	GraphViz	11
7	Conclusion	14

1. Introduction du problème

Ce projet porte sur l'implémentation de plusieurs algorithmes de recherche opérationnelle liés aux problèmes de flot dans les graphes orientés.

Un réseau de flot est un graphe orienté composé d'un ensemble de sommets et de deux sommets particuliers :

- la **source** notée s
- le **puits** noté t

Chaque arc (u, v) du graphe possède une capacité $c(u, v)$ représentant la quantité maximale de flot pouvant circuler de u vers v .

Un flot doit respecter les contraintes suivantes :

- **Contrainte de capacité :**

le flot circulant sur un arc ne peut pas dépasser sa capacité :

$$f(u, v) \leq c(u, v)$$

- **Contrainte de conservation :**

pour tout sommet intermédiaire, la quantité de flot entrant doit être égale à la quantité de flot sortant :

$$\sum f(u, v) = \sum f(v, u)$$

sauf pour la source s et le puits t .

La valeur d'un flot correspond à la quantité totale de flot quittant la source ou entrant dans le puits.

L'objectif principal est de maximiser cette valeur tout en respectant les contraintes du réseau.

Les algorithmes implémentés dans ce projet sont :

- Ford-Fulkerson pour le calcul du flot maximum et du min cut
- Min Cost Flow avec Bellman-Ford
- Min Cost Flow avec Dijkstra et renormalisation des coûts
- Détection de cycles négatifs

L'objectif de ces implémentations est de manipuler des graphes résiduels afin d'optimiser des flots sous contraintes de capacité et de coût.

2. Structure de données

2.1 Représentation du graphe

Dans ce projet, les graphes sont représentés sous forme de matrices d'adjacence. Ce choix a été fait car cette représentation est simple à manipuler et permet d'accéder rapidement aux capacités et aux coûts entre deux sommets.

Nous utilisons principalement trois matrices :

- une matrice de capacités
- une matrice de coûts
- une matrice de flot

La matrice de capacités $capacity[u][v]$ contient la capacité maximale de l'arc allant du sommet u vers le sommet v . Si aucun arc n'existe entre deux sommets, la valeur est égale à 0.

Par exemple :

$$capacity[0][1] = 20$$

signifie qu'il est possible d'envoyer au maximum 20 unités de flot du sommet 0 vers le sommet 1.

La matrice de coûts $cost[u][v]$ représente le coût associé au transport d'une unité de flot sur un arc. Cette matrice est principalement utilisée dans les algorithmes de *Min Cost Flow* afin de choisir les chemins les moins coûteux.

Enfin, la matrice $flow[u][v]$ permet de stocker la quantité de flot réellement envoyée sur chaque arête au cours de l'exécution des algorithmes.

Cette séparation entre capacités, coûts et flot permet de rendre le code plus clair et plus simple à mettre à jour pendant les différentes étapes des algorithmes.

Le choix d'utiliser des matrices d'adjacence simplifie également l'affichage et la visualisation du graphe avec Graphviz. Cependant, la matrice d'adjacence reste un choix assez coûteux en terme d'espace car il ne s'agit pas d'une structure creuse et donc utilise $O(N^2)$ en espace, N le nombre de sommets. Cette complexité spatiale est contrebalancée par la complexité temporelle de requête d'adjacence imbattable en $O(1)$.

2.2 Graphe résiduel

Le graphe résiduel est la structure principale utilisée dans les algorithmes de flot.

Il représente les capacités restantes du réseau après chaque augmentation de flot. À chaque fois qu'un chemin augmentant est trouvé, les capacités du graphe résiduel sont modifiées.

Pour chaque arête (u, v) :

- la capacité résiduelle diminue lorsque du flot est envoyé
- une arête inverse est ajoutée ou augmentée afin de permettre un éventuel retour de flot

La mise à jour du graphe résiduel est réalisée avec les opérations suivantes :

$$residuel[u][v]- = path_flow$$

$$residuel[v][u]+ = path_flow$$

Le principe de l'arête inverse est très important. Elle permet de corriger un choix effectué dans une itération précédente. En effet, un algorithme peut avoir envoyé du flot dans un chemin qui n'est finalement pas optimal. Grâce à l'arête inverse, il devient possible d'annuler une partie du flot et de le rediriger vers un meilleur chemin.

Le graphe résiduel est donc une représentation dynamique du réseau. Il évolue au fur et à mesure des itérations de l'algorithme jusqu'à ce qu'aucun nouveau chemin augmentant ne puisse être trouvé.

Cette structure est utilisée dans :

- Ford-Fulkerson pour le calcul du flot maximum
- Min Cost Flow pour l'optimisation des coûts
- la détection de cycles négatifs

3. Algo pour construire un flot max

3.1 Flot maximal et coupe minimale

3.1.1 Flot maximal

Le problème du flot maximal consiste à envoyer la plus grande quantité possible de flot depuis un sommet source s vers un sommet puits t tout en respectant les contraintes de capacité des arcs. Dans notre implémentation, le flot total est augmenté progressivement en cherchant des chemins augmentants dans le graphe résiduel.

À chaque itération :

- un chemin allant de la source au puits est trouvé
- on détermine la capacité minimale sur ce chemin
- cette valeur est ajoutée au flot total

La capacité minimale d'un chemin correspond au maximum de flot pouvant être envoyé sans dépasser les capacités des arcs. Le programme maintient également une matrice de flot :

$$flot[u][v]$$

Cette matrice permet de connaître la quantité exacte de flot circulant sur chaque arête du graphe. Lorsqu'un nouveau flot est envoyé sur un chemin augmentant, la mise à jour suivante est effectuée :

$$flot[u][v] + = path_flow$$

Le flot maximal final correspond à la somme totale des flots envoyés depuis la source vers le puits.

3.1.2 Min Cut

Le problème du *Min Cut* est directement lié au problème du flot maximal. Une coupe minimale correspond à un ensemble d'arêtes dont la suppression empêche toute communication entre la source et le puits. Après l'exécution de l'algorithme de Ford-Fulkerson, un parcours BFS est effectué dans le graphe résiduel. Les sommets atteignables depuis la source forment un ensemble noté S . Les sommets non atteignables forment un ensemble noté T . Les arêtes reliant un sommet de S vers un sommet de T constituent alors la coupe minimale du graphe. Le théorème max-flow/min-cut garantit que :

- la valeur du flot maximal
 - la capacité de la coupe minimale
- sont égales.

3.2 Recherche de chemin augmentant avec BFS

Le parcours BFS (*Breadth First Search*) est utilisé pour trouver un chemin augmentant entre la source et le puits dans le graphe résiduel. Le choix du BFS permet de parcourir le graphe niveau par niveau et de trouver rapidement un chemin disponible.

Dans notre implémentation :

- seuls les arcs ayant une capacité strictement positive sont explorés
- un tableau **parent** est utilisé afin de reconstruire le chemin trouvé
- un tableau **visited** permet d'éviter de visiter plusieurs fois le même sommet

Le BFS retourne :

- **True** si un chemin augmentant existe
- **False** sinon

Lorsque plus aucun chemin augmentant n'est trouvé, cela signifie que le flot maximal a été atteint.

3.3 Algorithme de Ford-Fulkerson

L'algorithme de Ford-Fulkerson est un algorithme classique permettant de calculer un flot maximal dans un réseau. Le principe général est de chercher de manière répétée des chemins augmentants dans le graphe résiduel jusqu'à ce qu'aucun nouveau chemin ne puisse être trouvé.

Le fonctionnement de l'algorithme est le suivant :

1. rechercher un chemin augmentant avec BFS
2. calculer la capacité minimale sur ce chemin
3. ajouter cette valeur au flot total
4. mettre à jour les capacités résiduelles
5. recommencer tant qu'un chemin existe

Dans notre implémentation, les capacités résiduelles sont mises à jour avec :

$$residual[u][v] - = path_flow$$

$$residual[v][u] + = path_flow$$

La première opération diminue la capacité restante sur l'arête directe. La seconde ajoute une capacité sur l'arête inverse afin de permettre un éventuel retour de flot dans les itérations suivantes. Cette propriété est essentielle car elle permet à l'algorithme de corriger certains choix effectués précédemment et d'obtenir une solution optimale. L'algorithme se termine lorsqu'aucun chemin augmentant n'existe entre la source et le puits.

3.3.1 Complexité

Avec une représentation matricielle du graphe la complexité temporelle :

$$O(NM^2)$$

et la complexité mémoire :

$$O(N^2)$$

où N représente le nombre de sommets et M représente le nombre d'arêtes

4. Min Cost Flow par Repeated Augmenting Paths

Le problème du *Min Cost Flow* consiste à envoyer un certain flot dans un réseau tout en minimisant le coût total du transport. Contrairement au problème du flot maximal, où l'objectif est uniquement de maximiser la quantité de flot envoyée, ici chaque arête possède également un coût associé.

L'objectif devient donc double :

- respecter les capacités des arcs
- minimiser le coût total du flot envoyé

Dans ce projet, l'algorithme utilisé repose sur la méthode des *Repeated Augmenting Paths*. Le principe consiste à chercher de manière répétée des chemins de coût minimal dans le graphe résiduel, puis à envoyer du flot sur ces chemins jusqu'à atteindre le flot demandé ou jusqu'à ce qu'aucun chemin valide ne puisse être trouvé.

4.1 Version avec Bellman-Ford et coûts négatifs

Dans les algorithmes de plus court chemin avec pondération, certains algorithmes comme Floyd-Warshall permettent de gérer des poids négatifs mais ne fonctionnent plus correctement en présence de circuits de poids strictement négatif. L'algorithme de Bellman-Ford¹ autorise la présence d'arêtes de coût négatif et permet également de détecter l'existence d'un cycle absorbant, c'est-à-dire un cycle dont le coût total est strictement négatif. À chaque itération :

- Bellman-Ford calcule le chemin augmentant de coût minimal
- du flot est envoyé sur ce chemin
- le graphe résiduel est mis à jour

Le graphe résiduel joue un rôle important dans l'algorithme. Lorsqu'un flot est envoyé sur une arête, une arête inverse est ajoutée afin de permettre une éventuelle correction du flot lors des itérations suivantes.

Le coût de cette arête inverse devient alors l'opposé du coût initial :

$$cost[v][u] = -cost[u][v]$$

4.1.1 Principe de Bellman-Ford

L'algorithme de Bellman-Ford permet de calculer les plus courts chemins depuis une source même lorsque le graphe contient des arêtes de poids négatifs.

1. https://fr.wikipedia.org/wiki/Algorithme_de_Bellman-Ford

Le fonctionnement général est le suivant :

1. initialiser les distances
2. répéter $n - 1$ fois le relâchement des arêtes
3. vérifier l'existence éventuelle de cycles négatifs

où n représente le nombre de sommets du graphe.

Le relâchement d'une arête consiste à vérifier si un chemin plus court peut être obtenu :

$$dist[v] > dist[u] + cost[u][v]$$

Si cette condition est vraie, alors la distance est mise à jour.

4.1.2 Cycles négatifs

Lors de la construction du graphe résiduel, des arêtes inverses sont ajoutées afin de permettre la modification ou l'annulation d'un flot déjà envoyé. Ces arêtes inverses possèdent un coût opposé :

$$cost[v][u] = -cost[u][v]$$

Même si le graphe initial ne contient que des coûts positifs, le graphe résiduel peut alors contenir des coûts négatifs. Cela peut créer des cycles négatifs, c'est-à-dire des cycles dont la somme des coûts est négative. Ces cycles sont problématiques car ils permettraient de diminuer le coût total indéfiniment en parcourant plusieurs fois le cycle. L'algorithme de Bellman-Ford permet de détecter ce type de situation.

4.1.3 Fonctionnement du Min Cost Flow

À chaque itération de l'algorithme :

- Bellman-Ford calcule le chemin de coût minimal
- le flot maximal possible est envoyé sur ce chemin
- les capacités résiduelles sont mises à jour
- les arêtes inverses sont ajoutées avec coût opposé

La mise à jour du graphe résiduel permet à l'algorithme de corriger certaines décisions précédentes et d'optimiser progressivement le coût total du flot.

Cette approche garantit l'obtention d'un flot de coût minimal tant qu'aucun cycle négatif n'est présent dans le graphe résiduel.

4.2 Avec Dijkstra et renormalisation des coûts

Dijkstra² ne peut pas être utilisé directement avec des coûts négatifs. Une renormalisation des coûts est donc effectuée grâce aux potentiels $h(v)$. Pour un arc (u, v) de coût $w(u, v)$, le coût réduit est :

$$w'(u, v) = w(u, v) + h(u) - h(v) \geq 0$$

Après chaque itération, les potentiels sont mis à jour par $h(v) \leftarrow h(v) + d(v)$ afin de garantir des coûts réduits positifs à l'itération suivante.

2. https://fr.wikipedia.org/wiki/Algorithme_de_Dijkstra

5. Détection de cycles négatifs

La détection des cycles négatifs repose sur Bellman-Ford. Après $|V| - 1$ relaxations, une relaxation supplémentaire est effectuée : si une distance peut encore être réduite, un cycle négatif existe.

On initialise `dist = [0] * n` plutôt qu'à $+\infty$ pour détecter les cycles dans toutes les composantes du graphe, sans avoir besoin d'une super-source.

Avant de lancer l'algorithme min-cost max-flow avec Dijkstra, on vérifie que le graphe ne contient pas de cycle négatif.

6. GraphViz

Graphviz¹ est utilisé afin de visualiser :

- le graphe initial
- le graphe résiduel
- le flot sur chaque arête

Exemple d’affichage :

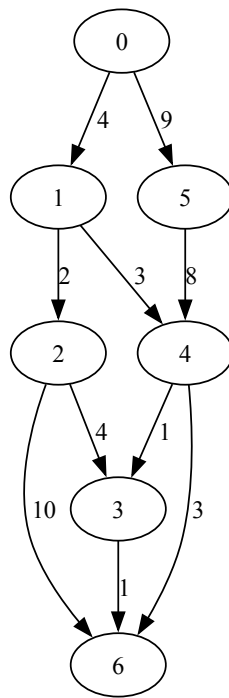


FIGURE 6.1 – Graphe initial

1. <https://graphviz.readthedocs.io/en/stable/manual.html>

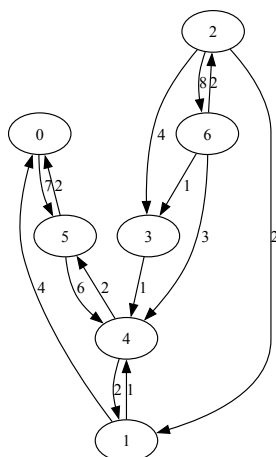


FIGURE 6.2 – Graphe résiduel après Ford-Fulkerson

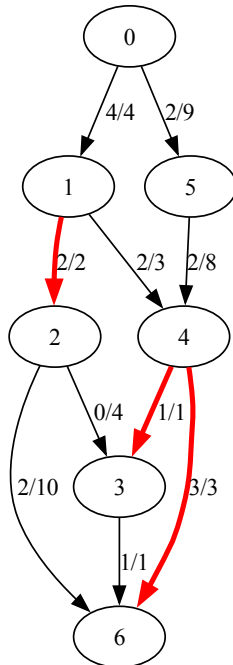


FIGURE 6.3 – Graphe flow-max/min-cut après Ford-Fulkerson

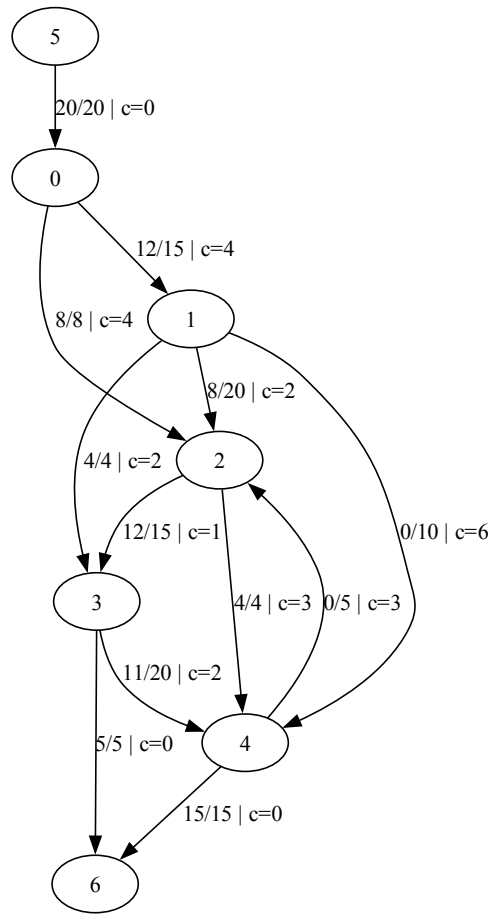


FIGURE 6.4 – Graphe min-cost-flow-max après BellmanFord

7. Conclusion

Ce projet a permis d'implémenter trois familles d'algorithmes classiques de recherche opérationnelle autour des problèmes de flot. Le graphe résiduel est la structure centrale de toutes ces approches : il permet de gérer les capacités restantes, de représenter les arêtes inverses et de corriger le flot à chaque itération. Les différentes méthodes étudiées :

- **Ford-Fulkerson** est simple à implémenter mais ne gère pas les coûts.
- **Bellman-Ford** supporte les coûts négatifs au prix d'une complexité plus élevée en $\mathcal{O}(NM)$.
- **Dijkstra** est plus efficace en pratique mais une renormalisation des coûts.

La détection de cycles négatifs s'est révélée indispensable : sans elle, les algorithmes de flot à coût minimum peuvent boucler indéfiniment ou produire des résultats incorrects.